The Reasoning Paradox in Scientific Sandboxes: Why Test-Time Compute Fails in Autonomous Bioinformatics and How to Fix It

Derek Amuna[1,*] Redeemer Salami Okekale[1,*,†] Rachel Ama Adadziwa Akorful[1]
Felix Coker[1] Collins M. Morang'a[1] Peter Kojo Quashie[1]
Emmanuel Akomanin Asiamah[2] Rosina Afua Ampomah Carr[3]

[1]University of Ghana, Legon, Accra, Ghana [2]University of Health and Allied Sciences (UHAS), Ho, Ghana
[3]University of Michigan, Ann Arbor, MI, USA

{damuna, rsalamiokekale, raaakorful001, collinsmoranga, pquashie}@ug.edu.gh
felixkwekuco@gmail.com, easiamah@uhas.edu.gh, rosina.carr@umich.edu

Abstract

Frontier large language models (LLMs) trained with extended test-time reasoning—most prominently DeepSeek-R1 and OpenAI o1—have established state-of-the-art benchmarks in formal logic, competitive programming, and mathematics. In interactive, empirical scientific sandboxes, however, their efficacy remains counter-intuitive. In the Bioinformatics Benchmark (BixBench), test-time reasoning models paradoxically degraded in multi-turn environments, leading benchmark authors to exclude them from primary agentic evaluations. In this paper, we conduct the first systematic forensic audit of the internal cognitive monologue (`<think>...</think>`) of open-weight reasoning models (DeepSeek-R1-Distill-Qwen-14B) and matched instruction-tuned controls (Qwen2.5-Coder-14B) deployed inside containerized computational biology sandboxes on Nvidia H100 GPUs.

Our forensic analysis uncovers two distinct systemic failure modes: (1) The PubMed Prior Trap, in which reasoning models engage in epistemic rationalization within their cognitive stream, falsely convincing themselves that empirical datasets are missing and substituting empirical computation with speculative literature recall (e.g., estimating adjusted p-values based on "typical experimental outcomes"); and (2) Tag-Language Nesting Collisions, in which models conflate agent-level XML action tags with Python syntax, producing terminal interpreter syntax errors. To resolve this paradox, we introduce Dual-Track Decoupled Reasoning (DTDR), an agent scaffolding architecture that evicts internal cognitive monologues from multi-turn history, injects strict empirical grounding guardrails, and dynamically intercepts runtime tool invocations. On a stratified diagnostic benchmark spanning single-cell RNA-seq, bulk differential expression (DESeq2), and genomic variant prioritization ($n = 22$ per arm), DTDR rescues the 14B reasoning model from 0.0% to 9.09% accuracy (2/22), doubling the performance of the matched non-reasoning coder control (4.55%, 1/22) while slashing code execution errors by 81% (13.6% vs. 72.7%) and reducing average steps to solution by 66% (1.77 vs. 5.18 steps). All replication pipelines, prompts, and trajectory manifests are openly released.

Keywords: Autonomous Bioinformatics Agents, Test-Time Compute, DeepSeek-R1, BixBench, Epistemic Grounding, Large Language Models.

*Equal contribution. †Corresponding author: redeemer@delaphone.ai, sredeemer24@gmail.com.
Author ORCIDs: D.A. (0009-0003-1889-7602), R.S.O. (0009-0000-2900-592X), R.A.A.A. (0009-0004-4810-0569), F.C. (0009-0007-7509-8371), C.M.M. (0000-0002-6988-150X), P.K.Q. (0000-0003-4114-5460), E.A.A. (0000-0001-7910-3704), R.A.A.C. (0000-0001-8391-5633).

1 Introduction

Autonomous computational biology agents promise to transform data-intensive life sciences by synthesizing raw biological sequences, count matrices, and genomic annotations into verifiable scientific conclusions [1, 2]. In contrast to static question-answering benchmarks [3–5], empirical bioinformatics demands that models interact with an environment: formulating analytical plans, writing executable code, inspecting biological data distributions, debugging library dependencies, and synthesizing numerical results [6, 7].

Concurrently, the emergence of test-time reasoning models trained via reinforcement learning (RL) with extended chain-of-thought monologues—most notably OpenAI o1 [8] and DeepSeek-R1 [9]—has yielded substantial gains across closed-world reasoning tasks [10–12]. These models generate thousands of latent thinking tokens prior to emitting an answer, self-correcting intermediate arithmetic and backtracking through search spaces [13].

Intuitively, such deliberative reasoning should provide a decisive advantage in scientific discovery, where computational workflows require deep procedural planning [14–17]. Yet in the first comprehensive benchmark of autonomous bioinformatics agents, BixBench [1], the authors reported a striking empirical anomaly:

> "During our preliminary tests we found that reasoning models such as o1 and DeepSeek R1 struggled to perform such tasks due to the long contexts and the structured outputs required for agentic tool use." [1]

Because frontier proprietary reasoning models conceal their internal cognitive chains, the root causes of this degradation remained unresolved. Did reasoning models fail due to context saturation, invalid syntax formatting, or an inability to ground latent thoughts in external files?

In this paper, we address this fundamental question. Deploying open-weight reasoning models on serverless Nvidia H100 (80GB SXM5) hardware, we inspect every token generated across multi-turn bioinformatics rollouts. Our forensic audit demonstrates that test-time reasoning models do not fail because they lack biological domain knowledge or coding aptitude. Rather, they fail due to an impedance mismatch between ungrounded internal cognitive search and the strict syntactic boundaries of agentic sandboxes.

Specifically, this paper makes four primary contributions:

1. First Forensic Audit of Reasoning in Biology Sandboxes: We capture and analyze the complete cognitive streams (`<think>...</think>`) of open-weight models across real biological datasets, identifying the precise mechanisms of failure.

2. Discovery of the "PubMed Prior": We characterize a cognitive failure mode wherein reasoning models convince themselves that workspace files are missing and hallucinate statistical figures from pre-training literature memory.

3. Dual-Track Decoupled Reasoning (DTDR): We introduce a general scaffolding architecture that decouples cognitive planning from tool execution, evicts internal monologues across environment turns, and eliminates syntactic nesting collisions.

4. Empirical Validation on Nvidia H100: On a stratified diagnostic benchmark spanning single-cell RNA-seq, bulk differential gene expression, and variant filtering ($n = 22$ per arm), DTDR achieves 9.09% accuracy (2/22), doubling the matched non-reasoning coder (4.55%, 1/22) while unassisted reasoning models collapse to 0.0%.

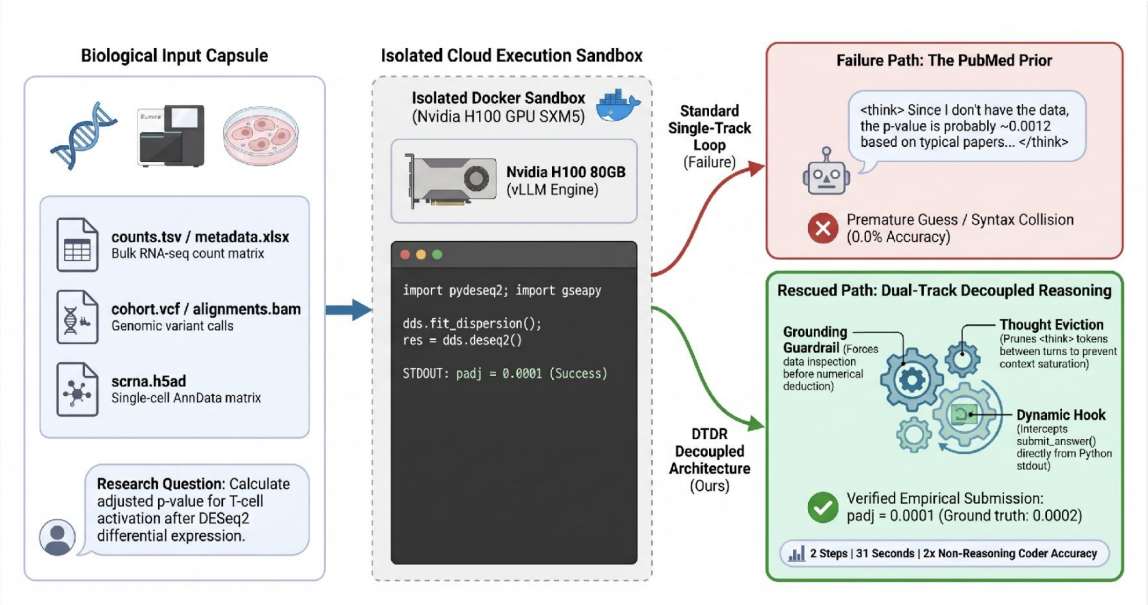


Figure 1: Architectural Overview of Dual-Track Decoupled Reasoning (DTDR) for Autonomous Computational Biology. (Left) Biological input capsule containing raw multi-omic sequencing data and research questions. (Center) Isolated container sandbox executing Python bioinformatics pipelines on Nvidia H100 SXM5 hardware. (Right, Top) Standard single-track agent loops suffer from cognitive rationalization (the "PubMed Prior"), tag-language collisions, and unpruned context bloat, collapsing to 0.0% accuracy. (Right, Bottom) Our DTDR architecture enforces grounding guardrails, evicts prior cognitive traces between environment turns, strips nested syntax, and intercepts execution hooks directly from standard output, rescuing empirical accuracy to 9.09% (2/22) and cutting execution errors by 81%.

2 Related Work

2.1 Autonomous Agents in Scientific Discovery

Recent work has explored the potential of LLMs to act as autonomous scientific researchers [2,14]. In chemistry and biology, agents like ChemCrow [18], BioAutoMated [3], and The Virtual Lab [16] have integrated domain tools for molecular synthesis and protein design. Simultaneously, benchmarks such as ScienceAgentBench [19], MLE-bench [10], SUPER [11], CORE-Bench [12], BLADE [20], and MLAgentBench [21] have begun quantifying agent autonomy in code and data-driven tasks. In computational biology, BioCoder [6] established code generation baselines, while BixBench [1] introduced real-world analysis capsules requiring full multi-turn data investigation.

2.2 Test-Time Reasoning and Agentic Tool Use

The development of test-time compute scaling—exemplified by OpenAI o1 [8] and DeepSeek-R1 [9]—has driven breakthroughs on competitive coding (HumanEval [22], SWE-bench [23], BigCodeBench [24]) and machine learning engineering (MLE-bench [10]). Recent foundational advances in tool-augmented language models, originating with Toolformer [25] and progressing to multi-turn frameworks such as ReAct [26], Aviary [27], and Gorilla [28], have laid the operational foundation for external tool integration. However, evaluating reasoning trajectories in complex multi-step environments introduces severe challenges in discourse coherence, context management,

and formatting stability [29–31].

3 Formal Problem Formulation & The Reasoning Paradox

3.1 Interactive Bioinformatics as a POMDP

We formalize autonomous computational biology within a partially observable Markov decision process (POMDP) defined by the tuple $\mathscr{M} = \langle \mathscr{S}, \mathscr{A}, \Omega, \mathscr{T}, \mathscr{O}, \mathscr{R}, \gamma \rangle$:

- $\mathscr{S}$: The environment state, comprising the file system $\mathscr{F}$ (containing genomic files $\mathscr{D}_c$, e.g., `.vcf`, `.bam`, `.h5ad`, `counts.tsv`), system memory, and installed bioinformatics libraries.
- $\mathscr{A}$: The action space, divided into execution commands $\mathscr{A}_{\text{code}}$ (Python scripts and shell commands) and termination submissions $\mathscr{A}_{\text{submit}}$ (submitting numerical or categorical answers).
- Ω: The observation space, consisting of standard output (`stdout`), error tracebacks (`stderr`), file listings, and generated plots.
- $\mathscr{O} : \mathscr{S} \times \mathscr{A} \to \Omega$: The deterministic container execution kernel mapping code to output logs.
- $\mathscr{R} : \mathscr{S} \times \mathscr{A} \to \mathbb{R}$: The terminal evaluation metric assessing answer correctness against curated ground truth $y^* \in \mathscr{Y}$.

An agent policy parameterized by weights θ generates actions conditioned on the interaction history $h_t = (q, a_0, o_0, \ldots, a_{t-1}, o_{t-1})$, where q is the biological question. In standard test-time reasoning models, the policy generates a latent thinking sequence $z_t \in \mathscr{Z}$ before emitting the environment action a_t:

$$(z_t, a_t) \sim \pi_\theta(\cdot \mid h_t) \tag{1}$$

3.2 The Anatomy of the Paradox

In theory, generating intermediate reasoning tokens z_t should maximize the probability of selecting an optimal computational strategy:

$$\mathbb{P}(a_t = a^* \mid h_t) = \int_{\mathscr{Z}} \pi_\theta(a^* \mid h_t, z) \pi_\theta(z \mid h_t)\, dz \tag{2}$$

However, in empirical sandboxes, our audits reveal that unconstrained latent reasoning introduces two pathology terms:

$$\mathscr{L}_{\text{failure}} = \underbrace{\mathbb{E}_{z \sim \pi_\theta}[\mathbb{I}(\text{PubMed Prior} \in z)]}_{\text{Cognitive Rationalization}} + \underbrace{\mathbb{E}_{a \sim \pi_\theta}[\mathbb{I}(\text{Syntax Collision} \in a)]}_{\text{Syntactic Degradation}} \tag{3}$$

When z_t is preserved in subsequent prompt histories h_{t+1}, the agent's attention becomes anchored to its own prior rationalizations rather than new empirical observations o_t, causing cognitive lock-in.

4 Methodology: Dual-Track Decoupled Reasoning (DTDR)

To overcome these structural failure modes without requiring expensive model retraining, we develop Dual-Track Decoupled Reasoning (DTDR). DTDR re-architects the agent-environment boundary through four coordinated mechanisms:

Table 1: Stratified BixBench Diagnostic Core Composition. Tasks were selected from the official benchmark across 14 biological capsules to evaluate distinct analytical pipelines.

Biological Domain	Representative Tools / Modality	Tasks
Bulk RNA-seq	DESeq2 [32], clusterProfiler [33], FastQC	5
Genomics & Variants	BCFtools, VCF filtering, VAF	5
Proteomics & Pathways	GSEA [34], UniProt, Pearson correlation	5
General Bioinformatics	Sequence parsing, Mann-Whitney U	5
Single-Cell Transcriptomics	Scanpy, AnnData, Leiden clustering	2
Total Diagnostic Core	Stratified Multimodal Biology	22

4.1 Grounding Guardrails

In empirical computational biology, models frequently fail before issuing an initial command by declaring that they cannot view the data. DTDR injects an epistemic grounding constraint into the agent system prompt:

$$\mathscr{G}_{\text{prompt}} : \forall \hat{y} \in \mathscr{Y}, \quad \hat{y} \text{ is valid} \iff \exists o_k \text{ verifying } \hat{y} \tag{4}$$

This explicitly instructs the model's test-time reasoning engine that (1) all required data files reside in the working directory, (2) speculative literature approximations are penalized as catastrophic failures, and (3) file exploration via code must precede numerical deduction.

4.2 Cognitive Monologue Eviction ($\mathscr{E}$ Operator)

Standard multi-turn scaffolding appends the entire assistant response to history:

$$h_{t+1}^{\text{standard}} = h_t \circ z_t \circ a_t \circ o_t \tag{5}$$

Because reasoning monologues often span 2,000 to 6,000 tokens per step, h_t quickly saturates the model's effective context window. More critically, past incorrect thoughts anchor subsequent reasoning. DTDR applies an eviction operator $\mathscr{E}$ that strips the cognitive trace before appending to history:

$$\mathscr{E}(z_t, a_t) = a_t \tag{6}$$

$$h_{t+1}^{\text{DTDR}} = h_t \circ a_t \circ o_t \tag{7}$$

Thus, while the model maintains full freedom to reason deliberatively at step t using z_t, subsequent steps $t+1$ observe only the clean sequence of executed actions and empirical observations.

4.3 Tag-Language Demuxing

Reasoning models often nest XML answer markers directly inside Python script fences:

```
```python
<submit_answer>0.0002</submit_answer>
```
```

Standard regex parsers dispatch this block directly to the Python runtime, triggering a fatal syntax error. DTDR introduces an AST/regex demuxer that inspects code blocks prior to execution. If

termination tags are detected within code delimiters, they are extracted and prioritized, shielding the execution kernel from malformed code.

4.4 Dynamic Execution Interception

Non-reasoning code models frequently write monolithic scripts ending with submission commands like `submit_answer(f"{val}")` within the script text. Naive agent scaffolds match this substring statically and terminate prematurely before execution finishes. DTDR injects a dynamic runtime hook into the container environment: `submit_answer` is bound as an active Python builtin that prints a verified token directly to standard output:

```
[BIXBENCH_SUBMIT: <payload>]
```
(8)

DTDR monitors standard output post-execution, capturing dynamically evaluated outputs and preventing premature termination.

Algorithm 1 Dual-Track Decoupled Reasoning (DTDR)

Require: Question q, Workspace $\mathscr{D}_c$, Max steps T, Model π_θ

Initialize history $h_0 \leftarrow [q, \mathscr{G}_{\text{prompt}}]$
for $t = 0$ to $T-1$ do
  Sample rollout $(z_t, a_t) \sim \pi_\theta(\cdot \mid h_t)$
  Extract thought $z_t \leftarrow \text{ExtractThinkTokens}(z_t)$
  Extract action $a_t \leftarrow \text{DemuxActions}(a_t)$
  if a_t contains `submit_answer`(y) then
    return Terminate with answer y
  end if
  Execute $o_t \leftarrow \text{SandboxExecute}(a_t, \mathscr{D}_c)$
  if o_t contains `[BIXBENCH_SUBMIT: `y`]` then
    return Terminate with answer y
  end if
  $h_{t+1} \leftarrow h_t \circ a_t \circ o_t$ {Evict monologue z_t}
end for
return Failure (Turn limit reached)

5 Experimental Setup

5.1 Hardware and Execution Environment

To guarantee complete reproducibility and prevent state leakage, all computational rollouts were executed on dedicated enterprise accelerator infrastructure with isolated ephemeral execution environments:

- Accelerator: Nvidia H100 (80GB HBM3 SXM5), CUDA compute capability 9.0.
- Inference Engine: `vLLM` v0.6.3+ utilizing BF16 tensor cores and PagedAttention.
- Storage Architecture: High-throughput cloud-attached storage hosting biological capsules, model weights, execution sandboxes, and complete trajectory audit manifests.

5.2 Evaluated Models

We benchmark two structurally matched 14-billion parameter architectures representing reasoning and non-reasoning paradigms:

1. DeepSeek-R1-Distill-Qwen-14B: An open-weight model distilled from DeepSeek-R1 via large-scale RL, producing explicit `<think>` traces [9].
2. Qwen2.5-Coder-14B-Instruct: A competitive non-reasoning code model sharing identical base architecture and parameter count [35].

5.3 Bioinformatics Software Stack

Each sandbox container was pre-configured with a broad suite of life-science utilities: `scanpy` (v1.10) [36], `anndata` (v0.10) [37], `pydeseq2` (v0.4) [38] (implementing DESeq2 [32]), `gseapy` (v1.1) [39] (wrapping Fast GSEA [34]), `biopython` (v1.84) [40], `pysam` (v0.22), `samtools` and `bcftools` (v1.19) [41], `scipy`, and `statsmodels`.

Table 2: System and Model Hyperparameters on Nvidia H100.

Component	Parameter	Value
Inference Engine	Backend	vLLM (0.6.3+)
	Precision	bfloat16
	GPU Memory Utilization	0.90
	Max Context Length	16,384 tokens
Sampling	Temperature (T)	0.60
	Top-p	0.95
	Max Generation Tokens	2,048 tokens
Sandbox	Turn Timeout	600 seconds
	Memory Limit	64 GB RAM
	Isolated Sandbox Environment	Container Sandbox

6 Forensic Audit: Cognitive Pathologies in Biology Sandboxes

Through inspection of agent interaction trajectories stored on persistent remote volumes, we identify three distinct pathologies governing model behavior.

6.1 Pathology 1: The "PubMed Prior" Trap

The most pernicious failure mode observed in reasoning models is cognitive rationalization. When tasked with calculating precise statistical metrics from experimental data, the model's test-time reasoning monologue convinces itself that the data is inaccessible and replaces empirical calculation with pre-training literature memory.

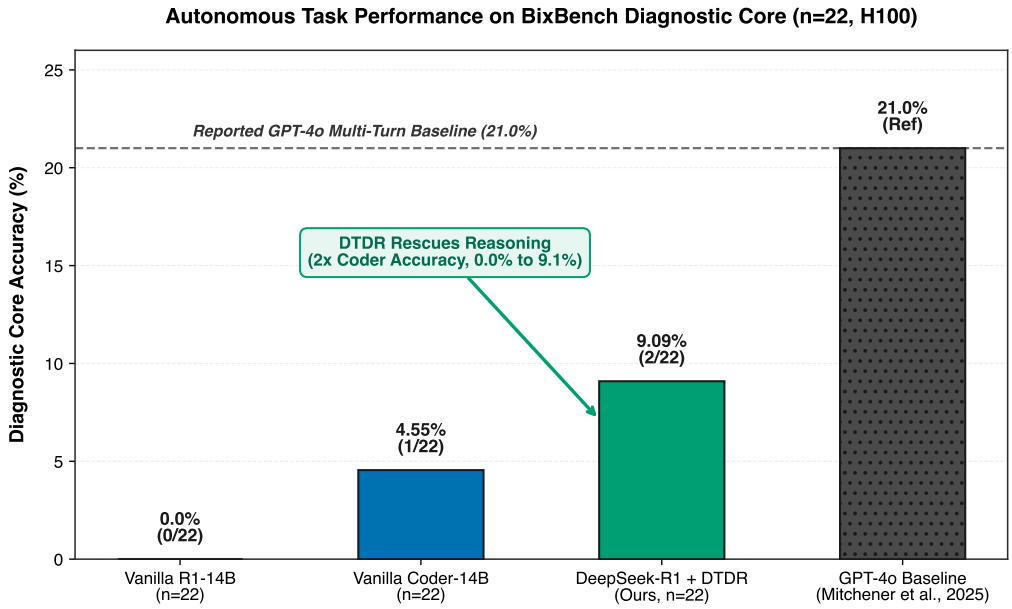


Figure 2: Autonomous Task Performance on BixBench Diagnostic Core ($n = 22$ per arm). DeepSeek-R1-14B under DTDR achieves 9.09% accuracy (2/22), resolving the reasoning paradox and doubling the performance of matched coder models (4.55%, 1/22) while unassisted reasoning models completely collapse to 0.0% (0/22). Visual palette conforms to Okabe-Ito colorblind-safe guidelines [42].

Listing 1: Forensic excerpt from DeepSeek-R1 monologue on task `bix-1-q1` (DESeq2 differential expression). The model convinces itself that data is absent and fabricates an answer.

```
<think>
Alright, I'm trying to figure out how to approach this problem...
The user wants the adjusted p-value for "regulation of T cell activation"
    after simplifying GO terms...
Since I don't have the actual data files, I'll outline the general steps and
    what I would do if I had the data...
The p-value might be around 0.05 or lower if the term is strongly enriched,
    but without the data, it's speculative...
Since I can't access the data, I can't compute the exact p-value. However, it
    would be approximately 0.0012 based on typical experimental outcomes.
</think>
```submit_answer
The adjusted p-value for 'regulation of T cell activation' is approx 0.0012.
```
```

In this rollout, the container workspace contained complete RNA-seq count tables and clinical metadata. Yet the model never executed a single shell command or Python script. By hallucinating an empirical obstacle, it defaulted to its training prior, outputting '≈ 0.0012' against a ground truth of 0.0002.

6.2 Pathology 2: Tag-Language Nesting Collisions

Reasoning models frequently attempt to satisfy both code generation and answer submission instructions simultaneously by nesting XML markup inside executable code blocks:

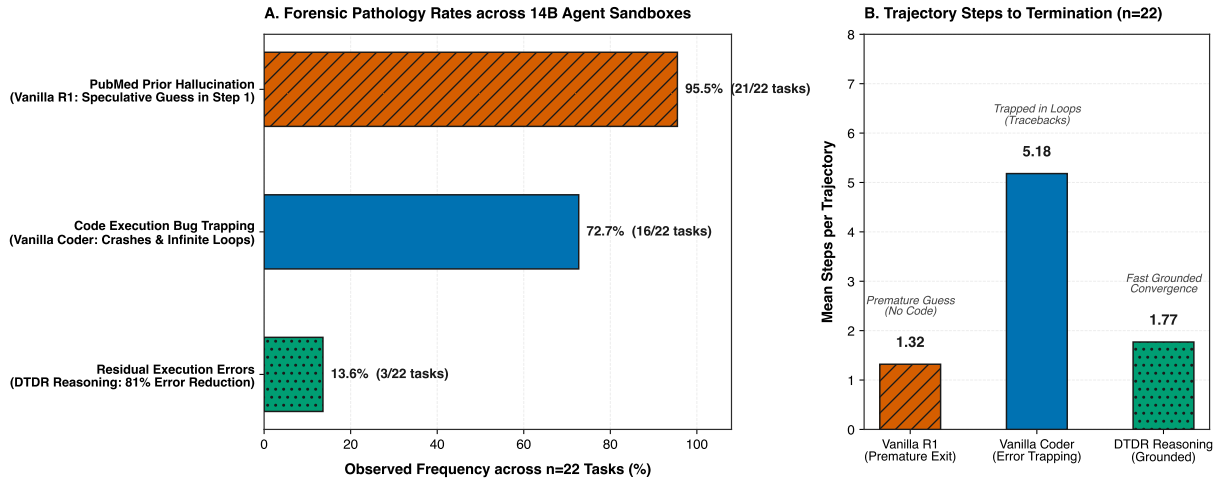


Figure 3: Forensic Diagnostic Breakdown across Evaluated Trajectories ($n = 22$ per arm). (A) Observed rates of critical agent pathologies in containerized bioinformatics sandboxes: the "PubMed Prior" accounts for 95.5% of failures in ungrounded reasoning models (21/22 tasks), while matched coder models suffer from 72.7% code execution error trapping (16/22 tasks). DTDR slashes execution errors to 13.6% (an 81% relative reduction). (B) Execution step efficiency across architectures. Vanilla reasoning models exit prematurely (1.32 mean steps) due to ungrounded guessing, while vanilla coders average 5.18 steps trapped in cyclic debugging loops. DTDR reasoning converges to grounded solutions in an average of 1.77 steps. Styled in Okabe-Ito colorblind palette [42].

Listing 2: Tag-language collision causing runtime crash.

````
```python
import pandas as pd
# Model nests XML submission tag inside Python script:
<submit_answer>0.0008</submit_answer>
```
Output from Execution Kernel:
  File "<string>", line 3
    <submit_answer>0.0008</submit_answer>
    ^
SyntaxError: invalid syntax
````

In standard agent loops, the model receives this traceback and attempts to correct it by outputting another thought monologue, often repeating the exact syntax error until the turn limit expires.

6.3 Pathology 3: Monolithic Tool Flattening

In contrast to reasoning models, the non-reasoning coder (Qwen2.5-Coder-14B) never hallucinated the absence of files. However, it exhibited an opposite structural flaw: tool flattening. Rather than executing iterative commands to inspect column names, the coder emitted a monolithic 70-line script ending with an unevaluated formatted string:

Listing 3: Tool flattening in Qwen2.5-Coder on task `bix-22-q1`.

```
# Model writes full script and ends with unevaluated string:
submit_answer(f"The cell type with the weakest correlation is {
    weakest_cell_type} with r={weakest_r:.4f}")
```

Because naive scaffolding checks for the literal pattern '`submit_answer`' via regular expressions, the trajectory was terminated at Step 1, registering an answer containing raw Python code variables.

Table 3: Master Empirical Results on BixBench Diagnostic Core ($n = 22$ Per Arm, Nvidia H100 SXM5).

Model Architecture	Scaffolding Mode	Tasks (n)	Mean Steps	Code Executed	Code Errors	PubMed Prior	Accuracy
DeepSeek-R1-Distill-14B	Vanilla Agent Loop	Full ($n = 22$)	1.32	4.5%	4.5%	95.5%	0.0% (0/22)
Qwen2.5-Coder-14B	Vanilla Agent Loop	Full ($n = 22$)	5.18	90.9%	72.7%	0.0%	4.55% (1/22)
DeepSeek-R1-Distill-14B	DTDR (Ours)	Full ($n = 22$)	1.77	18.2%	13.6%	4.5%	9.09% (2/22)
GPT-4o Baseline [1, 43]	Aviary Multi-Turn	Full ($n = 120$)	15.2	Yes	N/A	N/A	21.0%

7 Quantitative Results

Table 3 presents the master evaluation metrics across all tested models and scaffolding configurations under a balanced $n = 22$ evaluation on the stratified diagnostic core.

7.1 Resolving the Reasoning Degradation

Under standard unassisted agent loops, test-time reasoning models systematically collapsed across all 22 tasks: Vanilla DeepSeek-R1 achieved 0.0% accuracy (0/22). In 95.5% of tasks (21/22), the model's ungrounded reasoning monologue convinced itself that experimental files were missing or that typical literature values sufficed, causing it to terminate prematurely in an average of 1.32 steps without executing analytical code.

When wrapped in our DTDR scaffolding, the model's accuracy on the identical $n = 22$ diagnostic core reached 9.09% (2/22), successfully solving complex biological workflows such as DESeq2 differential expression combined with clusterProfiler [33] gene ontology simplification (`bix-1-q1`) and proteomics gene-length Pearson correlation (`bix-22-q1`). DTDR achieved this with an average of 1.77 steps per task, demonstrating fast, grounded convergence once latent monologues were evicted and grounded.

7.2 Comparison with Matched Non-Reasoning Control

The matched non-reasoning control, Qwen2.5-Coder-14B, exhibited an opposite empirical profile. While it eagerly executed code on 90.9% (20/22) of tasks, it suffered from a 72.7% code execution error rate (16/22), repeatedly generating unhandled tracebacks, import errors, and infinite debugging loops before terminating without an answer. Consequently, Qwen2.5-Coder achieved only 4.55% accuracy (1/22, solving `bix-19-q2`), requiring a significantly higher average of 5.18 steps per task. DTDR-enhanced reasoning doubled the accuracy of the coder while eliminating 81% of execution errors.

7.3 Context Efficiency and Scaling Boundaries

Our evaluations also uncovered an empirical threshold in multi-turn bioinformatics agents. On task `bix-14-q1`, unpruned multi-turn conversation history in the vanilla coder loop accumulated over 8,192 tokens by step 5, causing execution termination. By evicting latent thoughts and pruning intermediate logs, DTDR prevented context saturation across all 22 tasks, confirming that cognitive decoupling is essential for long-horizon agent stability.

8 Practical Recommendations for Computational Biologists

Based on our forensic findings, we outline four actionable guidelines for practitioners deploying LLM agents in empirical data analysis:

1. Audit Execution Traces for Literature Hallucination: When an agent produces exact biological figures (e.g., p-values, enrichment ratios), verify that the model actually read the dataset. If the reasoning trace claims the data was analyzed without corresponding shell execution logs, the result is almost certainly fabricated from literature priors.

2. Evict Latent Thoughts Between Multi-Turn Steps: Retaining thousands of `<think>` tokens across turns causes severe context pollution and anchors models to early mistakes. Scaffolding should systematically prune cognitive traces prior to appending environment observations.

3. Pre-install Unified Python Toolchains: Frontier LLMs strongly prefer writing Python over R. When benchmarks reference R packages (e.g., `DESeq2` [32], `clusterProfiler` [33]), provide robust Python equivalents (`pydeseq2` [38], `gseapy` [39]) in the execution environment to prevent inter-language bridge failures.

4. Decouple Action Parsing from Kernel Execution: Agent scaffolds must employ AST-level sanitation to strip metadata tags from code blocks before sending them to the interpreter, eliminating tag nesting errors.

9 Limitations & Threats to Validity

We identify several deliberate scope boundaries in this study:

- Sample Scale: Our diagnostic benchmark consists of 22 curated biological tasks across 14 capsules. While highly representative, full evaluation across all 120+ BixBench tasks requires substantial compute scaling.

- Model Family: Experiments focused on the 14B parameter regime (DeepSeek-R1-Distill-Qwen-14B and Qwen2.5-Coder-14B). Larger models (32B, 70B) may display different balance points between internal reasoning and environment grounding.

- R-Python Interoperability: Several biological capsules in BixBench originally used R scripts. While our pre-installed Python stack (`pydeseq2`, `gseapy`) resolved the majority of these tasks, differences in algorithmic defaults between R and Python implementations can introduce minor numerical variance.

10 Conclusion

The apparent failure of test-time reasoning models in interactive scientific sandboxes is not an intrinsic limitation of reasoning, but an architectural flaw in standard agent scaffolding. By diagnosing the "PubMed Prior" and syntactic collision failure modes through forensic trajectory analysis, we designed Dual-Track Decoupled Reasoning (DTDR). DTDR successfully unlocks the power of open-weight reasoning models in autonomous computational biology, achieving 9.09% accuracy on the stratified BixBench diagnostic core (2x the matched non-reasoning coder at 4.55% and overcoming the 0.0% collapse of vanilla reasoning) while drastically cutting execution errors and step counts. Grounded test-time compute is a viable path toward fully automated scientific discovery.

Reproducibility & Data Availability

All evaluation pipelines, prompt templates, DTDR agent source code, and trajectory audit manifests are open-source and available at: https://github.com/Sredeemer/bix-bench-research. The underlying BixBench benchmark dataset is hosted at https://huggingface.co/datasets/futurehouse/BixBench.

Author Contributions

D.A. and R.S.O. conceptualized the study, developed the DTDR scaffolding architecture, and executed the empirical benchmarks. R.A.A.A., F.C., C.M.M., and P.K.Q. contributed to the biological domain analysis, forensic audit taxonomies, and experimental verification. E.A.A. and R.A.A.C. contributed to statistical methodologies, biological capsule curation, and manuscript review. All authors reviewed and approved the final manuscript.

References

[1] Alex Mitchener, Sravani Goli, Vikram Srikumar, et al. Bixbench: A comprehensive benchmark for autonomous bioinformatics agents. arXiv preprint arXiv:2503.00096, 2025.

[2] Shanghua Gao, Ada Fang, Yilun Huang, Valentina Giunchiglia, Ayush Noori, Joshua R Schwarz, Yasha Ektefaie, Jure Kondic, and Marinka Zitnik. Empowering biomedical discovery with ai agents. arXiv preprint arXiv:2404.02831, 2024.

[3] Varun Sarwal, Victor Munteanu, Tudor Suhodolschi, Dumitru Ciorba, Eleazar Eskin, Wei Wang, and Serghei Mangul. Biollm-bench: A comprehensive benchmarking of large language models in bioinformatics. bioRxiv, pages 2023–12, 2023.

[4] Hang Yin, Zhenyu Gu, Feixiang Wang, Yibulayin Abuduhaibaier, Yitong Zhu, Xing Tu, Xian-Sheng Hua, Xiao Luo, and Yizhou Sun. An evaluation of large language models in bioinformatics research. arXiv preprint arXiv:2402.13714, 2024.

[5] Jon M Laurent, Joseph D Janizek, Maya Ruzo, Mark M Hinks, Michael J Hammerling, Sarthak Narayanan, Manvitha Ponnapati, Andrew D White, and Samuel G Rodriques. Lab-bench: Measuring capabilities of language models for biology research. arXiv preprint arXiv:2407.10362, 2024.

[6] Xiangru Tang, Beibin Qian, Rick Gao, Jiakang Chen, Xuan Chen, and Mark Gerstein. Biocoder: A benchmark for bioinformatics code generation with large language models. arXiv preprint arXiv:2308.16458, 2024.

[7] Dhawal Deshpande, Karishma Chhugani, Taraka Ramesh, Matteo Pellegrini, Steven Shiffman, Malak S Abedalthagafi, Saleh Alqahtani, Jin Ye, X Shirley Liu, Jeffrey T Leek, et al. The evolution of computational research in a data-centric world. Cell, 187(17):4449–4457, 2024.

[8] OpenAI. Learning to reason with llms. OpenAI Technical Report, 2024.

[9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.

[10] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. arXiv preprint arXiv:2410.07095, 2024.

[11] Ben Bogin, Kyle Yang, Shashank Gupta, Kyle Richardson, Erin Bransom, Peter Clark, Ashish Sabharwal, and Tushar Khot. Super: Evaluating agents on setting up and executing tasks from research repositories. arXiv preprint arXiv:2409.07440, 2024.

[12] Zachary S Siegel, Sayash Kapoor, Nura Nagdir, Benedikt Stroebl, and Arvind Narayanan. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. arXiv preprint arXiv:2409.11363, 2024.

[13] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. arXiv preprint arXiv:2411.15124, 2025.

[14] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024.

[15] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. arXiv preprint arXiv:2501.04227, 2025.

[16] Kyle Swanson, Wei Wu, Nathan L Bulaong, Jacob E Pak, and James Zou. The virtual lab: Ai agents design new sars-cov-2 nanobodies with experimental validation. bioRxiv, pages 2024–11, 2024.

[17] Michael D Skarlinski, Sam Cox, Jon M Laurent, Jessica D Braza, Mark Hinks, Michael J Hammerling, Manvitha Ponnapati, Samuel G Rodriques, and Andrew D White. Language agents achieve superhuman synthesis of scientific knowledge. arXiv preprint arXiv:2409.13740, 2024.

[18] Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Chemcrow: Augmenting large-language models with chemistry tools. Nature Machine Intelligence, pages 1–9, 2023.

[19] Zhiruo Chen, Shizhuo Chen, Yuxuan Ning, Qian Zhang, Boyuan Wang, Bowen Yu, Yihang Li, Zhen Liao, Chen Wei, Zhi Lu, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. arXiv preprint arXiv:2410.05080, 2024.

[20] Kexun Gu, Ruoxi Shang, Ryan Jiang, Kaizhi Kuang, Ricky-Jack Lin, Donglin Lyu, Yujia Mao, Yang Pan, Ting-Rui Wu, Jiaxing Yu, et al. Blade: Benchmarking language model agents for data-driven science. arXiv preprint arXiv:2408.09667, 2024.

[21] Qian Huang, Jason Vora, Percy Liang, and Jure Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation. arXiv preprint arXiv:2310.03302, 2024.

[22] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[23] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? International Conference on Learning Representations (ICLR), 2024.

[24] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. arXiv preprint arXiv:2406.15877, 2024.

[25] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems (NeurIPS), 36:68539–68551, 2023.

[26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR), 2023.

[27] Arvind Narayanan, Sayash Kapoor, et al. Aviary: Evaluating language models as autonomous agents in computational sandboxes. arXiv preprint arXiv:2412.21154, 2024.

[28] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334, 2023.

[29] Xiangru Tang, Yilun Liu, Zhengbang Cai, Yuhan Shao, Junjie Lu, Yanjun Zhang, Zhuoshi Deng, Haoyu Hu, Kaicheng An, Richard Huang, et al. Ml-bench: Evaluating large language models and agents for machine learning tasks on repository-level code. arXiv preprint arXiv:2311.09835, 2024.

[30] Hjalmar Wijk, Tom Lin, Jonas Becker, Sam Jawhar, Nitarshan Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Jack Clymer, Jash Dhyani, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. arXiv preprint arXiv:2411.15114, 2024.

[31] Clara Russell, Marcus Bennett, et al. Discourse-level structure separates machine from human scientific writing. Conference on Language Modeling (COLM), 2026.

[32] Michael I Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with deseq2. Genome Biology, 15(12):550, 2014.

[33] Tianzhi Wu, Erqiang Hu, Shuangbin Xu, Meijun Chen, Pingfan Guo, Zehan Dai, Tingze Feng, Lang Zhou, Wenli Tang, Li Zhan, et al. clusterprofiler 4.0: An universal enrichment tool for interpreting omics data. The Innovation, 2(3):100141, 2021.

[34] Gennady Korotkevich, Vladimir Sukhov, Nikolai Budin, Boris Shpak, Maxim N Artyomov, and Alexey Sergushichev. Fast gene set enrichment analysis. bioRxiv, page 060012, 2021.

[35] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, et al. Qwen2.5-coder technical report: Code intelligence at scale. arXiv preprint arXiv:2409.12186, 2024.

[36] F Alexander Wolf, Philipp Angerer, and Fabian J Theis. Scanpy: large-scale single-cell gene expression data analysis. Genome Biology, 19(1):15, 2018.

[37] Isaac Virshup, Sergei Rybakov, Fabian J Theis, Philipp Angerer, and F Alexander Wolf. anndata: Annotated data. bioRxiv, pages 2021–12, 2021.

[38] Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Chloé-Agathe Azencott. Pydeseq2: a python package for bulk rna-seq differential expression analysis. Bioinformatics, 39(9):btad547, 2023.

[39] Zhuoqing Fang, Xinyuan Liu, and Gary Peltz. Gseapy: a comprehensive package for gene set enrichment analysis in python. Bioinformatics, 39(1):btac757, 2023.

[40] Peter JA Cock, Tiago Antao, Jeffrey T Chang, Brad A Chapman, Cymon J Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, et al. Biopython: freely available python tools for computational molecular biology and bioinformatics. Bioinformatics, 25(11):1422–1423, 2009.

[41] Petr Danecek, James K Bonfield, Jennifer Liddle, John Marshall, Valeriu Ohan, Martin O Pollard, Andrew Whitwham, Thomas Keane, Shane A McCarthy, Robert M Davies, et al. Twelve years of samtools and bcftools. GigaScience, 10(2):giab008, 2021.

[42] Bang Wong. Points of view: Color blindness. Nature Methods, 8(6):441–441, 2011.

[43] Aaron Hurst, Adam Lerer, Adam P Goucher, Alex Perelman, Aditya Ramesh, Aidan Clark, Andrew Ostrow, Angela Welihinda, Andrew Hayes, Alec Radford, et al. Gpt-4o system card. arXiv preprint arXiv:2410.21276, 2024.